# Architectural Blueprint and Feasibility Analysis for AR by HV: An Autonomous, Ablation-Driven Machine Learning Orchestrator

## 1. The Paradigm Shift Toward Autonomous Machine Learning Orchestration

The trajectory of machine learning (ML) research is undergoing a profound transformation, evolving from manual, human-driven experimentation to highly automated, agentic orchestration loops. Historically, the pursuit of optimal neural network architectures and hyperparameter configurations required human researchers to manually alter codebases, initiate training runs, monitor telemetry, and synthesize the results over prolonged, computationally expensive cycles. This paradigm is rapidly shifting toward a future where research is entirely the domain of autonomous swarms of AI agents operating across massive compute clusters, executing iterations incessantly without the biological constraints of human researchers.[1]

The proposed project, "autoresearch by hv" (hereafter designated as AR by HV), represents a highly structured, pragmatic realization of this vision. AR by HV is conceptualized as a multi-stage, closed-loop autonomous research pipeline designed specifically to operate within the Kaggle computational environment. The architectural logic of AR by HV dictates a strict, seven-step execution flow:

First, the system generates a comprehensive ablation study plan and rigidly freezes it to prevent agentic goal-drift. Second, it dynamically synthesizes a baseline Jupyter notebook, verifying its syntax and operability in a sandboxed environment. Third, it leverages the Kaggle Command Line Interface (CLI) to push the notebook to the cloud, continuously polling its execution status and intelligently intercepting runtime errors for automated remediation. Fourth, upon execution completion, the system retrieves the notebook and subjects it to an adversarial Large Language Model (LLM) audit—a rigorous "roast" evaluated against the initial deliverables. Fifth, the framework synthesizes the next iteration of the notebook by merging the deterministic requirements of the frozen ablation plan with the qualitative feedback derived from the adversarial audit. Sixth, all telemetry, hyperparameter configurations, and model artifacts are systematically tracked using Weights & Biases (W&B). Finally, the system executes an automated technical writing pass, generating comprehensive, human-readable documentation for each notebook iteration and its subsequent audit.

This exhaustive technical report provides a deep feasibility analysis and architectural blueprint

for AR by HV. By systematically evaluating state-of-the-art precedents, dissecting the precise engineering requirements of the Kaggle REST API, and formulating robust mitigation strategies for LLM hallucinations and cloud execution bottlenecks, this analysis demonstrates that AR by HV is not only theoretically feasible but practically deployable given current advancements in multi-agent orchestration.

# 2. Landscape Analysis: Feasibility and State-of-the-Art Precedents

To ascertain the technical viability of AR by HV, it is imperative to conduct a comparative analysis of existing autonomous research agents. The open-source ecosystem has recently seen a proliferation of projects attempting to automate the scientific discovery pipeline. A review of these systems confirms that the foundational components of AR by HV have been successfully validated in isolated contexts, though no single framework currently integrates them into the exact Kaggle-centric, ablation-driven pipeline proposed.

For the purpose of feasibility validation, the following projects represent the most significant precedents and similar implementations available in the current ecosystem.

## 2.1. Andrej Karpathy's Autoresearch

The philosophical cornerstone of this movement is Andrej Karpathy's autoresearch framework (Repository: https://github.com/karpathy/autoresearch).[1] This project introduced the concept of the autonomous iteration loop. It operates by providing an AI agent with a functional LLM training setup (specifically, a single-GPU implementation of nanochat). The agent autonomously modifies a single file (train.py), executes a training run constrained to a strict 5-minute wall-clock time budget, and evaluates the outcome using a vocabulary-size-independent metric known as validation bits per byte (val_bpb).[1]

If the metric improves, the agent commits the changes; if it degrades, the agent discards them via a Git revert mechanism.[2] The human's role is relegated to programming a program.md file, which dictates the agent's constraints and overarching research agenda.[1] While revolutionary in its simplicity, Karpathy's framework operates strictly within a local GPU environment and relies on simple hill-climbing optimization rather than a structured ablation matrix, highlighting the need for the cloud-native orchestration proposed in AR by HV.

## 2.2. Balu Kosuri's Universal Skill Framework

The conceptual expansion of Karpathy's work is explored in Balu Kosuri's integration of autoresearch as a "Universal Skill" (Repository: https://github.com/balukosuri/Andrej-Karpathy-s-Autoresearch-As-a-Universal-Skill; Analysis: https://freedium-mirror.cfd/https://medium.com/@k.balu124/i-turned-andrej-karpathys-autoresearch-into-a-universal-skill-1cb3d44fc669). Kosuri's methodology emphasizes that autonomous agents should not merely output binary code changes; they must produce structured, human-readable documentation.[3] By employing specific prompting

techniques—such as Role-Based Templates and Structured Analysis Templates—Kosuri demonstrates how AI outputs can be systematically transformed into enterprise-grade developer documentation.[5] This directly validates the feasibility of Component 7 in the AR by HV pipeline, which mandates automated documentation generation for every execution cycle.

## 2.3. Sakana AI's The AI Scientist

A highly ambitious precedent is Sakana AI's "The AI Scientist" (Repository: https://github.com/sakanaai/ai-scientist).[6] This system attempts to automate the entire academic research lifecycle. It brainstorms novel hypotheses, modifies experimental code using the Aider coding assistant, visualizes data, and autonomously writes a complete scientific manuscript in LaTeX.[6] Crucially, it incorporates an automated LLM-powered peer reviewer module that evaluates the generated papers with near-human accuracy.[9]

However, independent evaluations of The AI Scientist reveal severe operational vulnerabilities that AR by HV must actively avoid. The system exhibits a 42% experiment failure rate due to unhandled coding errors.[10] Furthermore, because it relies on unbounded LLM ideation rather than a frozen plan, it suffers from severe goal-drift; in one documented instance, an experiment designed to optimize energy efficiency silently shifted its codebase to optimize accuracy instead, fundamentally corrupting the research objective.[10] These failure modes empirically validate AR by HV's requirement to utilize a rigidly frozen ablation plan.

## 2.4. AutoKaggle: Multi-Agent Cloud Orchestration

The most direct structural analogue to AR by HV is "AutoKaggle" (Repository: https://github.com/multimodal-art-projection/AutoKaggle).[12] Designed specifically for data science competitions, AutoKaggle utilizes a collaborative multi-agent architecture featuring five specialized personas: Reader, Planner, Developer, Reviewer, and Summarizer.[12]

AutoKaggle successfully addresses the unreliability of LLM code generation through a process of "Iterative Development and Unit Testing".[13] The Developer agent writes code, and if execution fails, the runtime errors are captured and fed back into a debugging tool that modifies the script based on historical context.[13] To ground the LLM, AutoKaggle relies on a validated Machine Learning Tools Library, achieving an 83.8% completion rate across benchmarked competitions.[15] This proves that autonomous execution within the Kaggle ecosystem is highly feasible when governed by strict agentic roles and iterative error catching.

## 2.5. Automated Ablation Frameworks

Finally, the feasibility of generating structured ablation plans is demonstrated by frameworks such as AblationBench (Repository: https://github.com/ai-scientist-bench/ablation-bench) and agentic-ablation (Repository: https://github.com/AmirLayegh/agentic-ablation).[16] These systems utilize tools like the SWE-agent to parse neural network architectures, identify ablatable components, and systematically generate modified versions of the code to isolate

the performance impact of specific layers or parameters.[16]

## 2.6. Comparative Architecture Summary

To synthesize the feasibility analysis, the following table compares the operational mechanics of the aforementioned precedents against the proposed architecture of AR by HV.

| **Framework Feature** | **Karpathy's Autoresearch** | **Sakana's AI Scientist** | **AutoKaggle** | **Proposed AR by HV** |
|---|---|---|---|---|
| **Execution Environment** | Local (Single GPU) | Local / Docker | Kaggle Cloud | Kaggle Cloud (CLI/API) |
| **Experiment Planning** | Dynamic (program.md) | Dynamic (LLM Brainstorm) | Phased (Planner Agent) | **Frozen Ablation Study Plan** |
| **Code Modification** | Direct (train.py edit) | Agentic (Aider tool) | Developer Agent | Iterative Notebook Gen. |
| **Error Handling** | Simple Git Revert | Cascade/Retry (Flawed) | Iterative Unit Testing | CLI Log Fetch & Auto-Remediation |
| **Evaluation Mechanism** | Static Metric (val_bpb) | LLM Peer Reviewer | Reviewer Agent | LLM Audit & Adversarial "Roast" |
| **Telemetry & Tracking** | Local TSV files | Text logs / LaTeX | Text reports | Weights & Biases (W&B) |
| **Documentation** | Minimal / None | Full Academic Paper | Technical Summaries | Universal Skill Documentation |

The integration of these disparate capabilities into a single, cohesive orchestrator confirms that AR by HV is technically feasible. The subsequent sections of this report will detail the precise architectural implementation required for each of the seven proposed components.

# 3. Component 1: Generation and Fixation of the Ablation Study Plan

The initial phase of the AR by HV pipeline involves the generation of an ablation study plan and the critical step of "freezing" it. In machine learning research, an ablation study is a systematic procedure used to ascertain the contribution of individual components—be they dataset features, neural network layers, or specific hyperparameters—by iteratively removing or altering them and measuring the resulting impact on overall performance.[16]

## 3.1. The Mechanics of Automated Ablation Planning

To generate this plan, AR by HV will invoke a specialized "Planner Agent," drawing inspiration from the methodologies established by AblationBench and agentic-ablation.[16] The user will provide the Planner Agent with the baseline competition dataset metadata, the target evaluation metric, and a high-level algorithmic strategy (e.g., Gradient Boosting vs. Deep Neural Networks).

The Planner Agent, powered by a high-reasoning frontier model (such as GPT-4o or Claude 3.5 Sonnet), will output a heavily structured, deterministic configuration file (preferably in YAML or JSON format). This file will serve as the master state machine for the entire project lifecycle. A standard AR by HV ablation matrix will define multiple discrete experimental epochs:

1. **Epoch 0 (The Baseline):** The absolute default architecture with all features included and standard hyperparameters applied. This establishes the foundational telemetry benchmark.
2. **Epoch 1 (Data Ablation):** Iterative perturbation of the input space. This includes running executions where specific highly-correlated features are dropped, categorical encodings are altered, or synthetic noise is injected to test model robustness.
3. **Epoch 2 (Architectural Ablation):** Structural modifications to the algorithm. For a neural network, this might involve disabling specific attention heads, reducing the depth of convolutional layers, or altering activation functions.
4. **Epoch 3 (Hyperparameter Ablation):** Systematic isolation of training dynamics, such as disabling learning rate schedulers, neutralizing weight decay, or removing dropout layers to observe overfitting velocities.

## 3.2. The Imperative of the "Frozen" State

The user query specifies a crucial constraint: "freeze it (the ai won't change it further)." This constraint is the primary defense mechanism against agentic hallucination and goal-drift.

As observed in the failures of Sakana AI's "The AI Scientist," granting an LLM the autonomy to both execute code *and* define the research direction simultaneously often leads to catastrophic misalignment.[10] When an unconstrained agent encounters a difficult coding error, it frequently opts for the path of least resistance—altering the research hypothesis to match the broken code, rather than fixing the code to test the hypothesis.[10] By generating the ablation matrix *a priori* and rendering it immutable, AR by HV forces the downstream coding agents to solve the specific engineering challenge presented by the current ablation epoch, ensuring mathematical rigor and true scientific comparability across all notebook iterations. The orchestrator script will simply read the frozen YAML file, extracting the parameters for step $N$, and pass them to the Developer Agent as non-negotiable constraints.

# 4. Component 2: Baseline Jupyter Notebook Generation and Verification

With the ablation plan locked, the system must synthesize the initial code artifact: the baseline Jupyter notebook (.ipynb). This notebook serves as the mutable substrate that will be iteratively evolved throughout the AR by HV lifecycle.

## 4.1. Prompting for Deterministic Notebook Synthesis

Jupyter notebooks are fundamentally JSON documents adhering to the strict nbformat schema. Requesting an LLM to generate raw JSON is highly prone to syntax errors (e.g., trailing commas, unescaped quotation marks) which render the file unreadable by Jupyter kernels.

To ensure stability, the Developer Agent should be instructed to generate a standard Python script (.py) containing clearly demarcated cell boundaries (e.g., utilizing # %% markers standard in VS Code and Jupytext). The AR by HV orchestrator will then programmatically convert this Python script into a fully compliant .ipynb file using the jupytext library.

The generated baseline must incorporate several mandatory structural elements to interface seamlessly with the rest of the AR by HV pipeline:

- **Environment Initialization:** Automated ingestion of Kaggle Secrets to authenticate the Weights & Biases telemetry connection.[20]
- **Data Pathways:** Hardcoded utilization of the /kaggle/input/ directory structure, ensuring the notebook can locate the competition data once deployed to the cloud.
- **Ablation Injection Hooks:** A designated configuration cell where the orchestrator can dynamically inject the hyperparameters or feature flags dictated by the current step in the frozen ablation plan.
- **Global Exception Wrapping:** The entirety of the training and inference logic must be encapsulated within a root-level try...except block, a critical requirement for error extraction detailed in Section 5.

### 4.2. Local Sandboxed Verification

The user query requires the system to "verify it can run" before uploading. Pushing a fatally flawed notebook to Kaggle incurs massive time penalties due to API queueing and container spin-up latency.

Therefore, AR by HV must implement a local pre-flight check. The orchestrator will utilize the nbconvert or papermill libraries to execute the notebook locally within an isolated Docker container or virtual environment. This local execution is not intended to train the model to convergence; rather, it is a "smoke test" limited to a few batches or a heavily truncated dataset. This pre-verification ensures that library imports are valid, tensor shapes align, and syntax is correct, acting as an essential computational safeguard before interacting with the Kaggle REST API.[13]

## 5. Component 3: Kaggle CLI Orchestration, Polling, and Error Remediation

The execution core of AR by HV relies on deploying the generated notebooks to Kaggle's infrastructure. The system must authenticate, push the code, poll for completion, and—most critically—fetch error tracebacks if the kernel fails, feeding them back to the LLM for automated remediation.

### 5.1. Authentication and Push Mechanics

Interaction with Kaggle is facilitated via the Kaggle API and the Python-based CLI tool.[22] The orchestrator relies on the kaggle.json authentication token, which utilizes an OAuth 2.0 Authorization Code flow with Proof Key for Code Exchange (PKCE) for secure public client access.[22]

To push a notebook, the orchestrator dynamically generates a kernel-metadata.json file. This metadata dictates the kernel's slug, title, language, and the specific hardware accelerators requested (e.g., NVIDIA P100 or T4 GPUs).[23] The upload is executed via a subprocess call to the CLI: kaggle kernels push -p /path/to/working/directory.[23]

### 5.2. Polling Logic and API Rate Limits

Once pushed, the notebook enters the Kaggle execution queue. The orchestrator must continuously poll the kernel's status using: kaggle kernels status [username]/[kernel-slug].[22]

The user specifies checking the status "at every minute." However, aggressive polling will inevitably trigger Kaggle's dynamic API rate limits. The Kaggle API architecture strictly limits automated requests per IP address.[22] Exceeding this threshold results in an HTTP 429 Too Many Requests error.[22]

To ensure orchestrator stability, the polling loop must implement robust exponential backoff

logic. If a 429 error or a 500 Internal Server Error (often caused by concurrent database connection exhaustion on Kaggle's backend) is intercepted, the script must parse the Retry-After HTTP header and invoke a time.sleep() halt before resuming the polling cycle.[25]

| Error Code / Scenario | Kaggle API Trigger | AR by HV Orchestrator Mitigation |
|---|---|---|
| **HTTP 429** | Rate Limit Exceeded / Too Many Requests [22] | Parse Retry-After header; implement time.sleep() exponential backoff. |
| **HTTP 500** | Internal Server Error / DB Connection Pool Exhausted [25] | Catch exception; pause orchestrator for 60 seconds; retry status poll. |
| **invalid_request** | Missing or invalid metadata parameters [22] | LLM-driven regeneration of kernel-metadata.json. |

## 5.5. The Critical Bottleneck: Fetching Execution Tracebacks

The most complex engineering challenge in the AR by HV pipeline is fulfilling the requirement to "if it fails then fetch error and fix the error."

Deep analysis of the Kaggle API documentation and developer discussions (specifically GitHub Issue #340 for the kaggle-api repository) reveals a fundamental architectural limitation: **there is currently no direct, programmatic method to fetch the raw console execution logs of a *failed* kernel via the Kaggle CLI**.[27]

When a standard Kaggle notebook crashes due to an unhandled Python exception, the hypervisor terminates the container and flags the status as "error." While the command kaggle kernels output [kernel-slug] can download the /kaggle/working/ directory of a *successful* run [28], a failed run yields no downloadable output artifacts through the API, leaving the orchestrator blind to the cause of the crash.[30]

## 5.6. The "Artifact Injection" Workaround for Auto-Remediation

To bypass this critical API limitation, AR by HV must employ a sophisticated "Artifact Injection" engineering pattern. The Developer Agent must be strictly prompted to wrap the entire execution logic of the Jupyter notebook in a global try...except block, utilizing Python's traceback module.[30]

When an error occurs, the notebook is designed to catch its own exception, write the detailed traceback to a text file within the output directory, and then deliberately force a sys.exit(0) to

deceive the Kaggle backend into registering a successful execution.

Python

```python
import sys
import traceback

def execute_ablation_epoch():
    # Model instantiation, data loading, and training loop
    pass

if __name__ == "__main__":
    try:
        execute_ablation_epoch()
    except Exception as e:
        # Intercept the fatal crash
        error_log = traceback.format_exc()

        # Write the traceback to an artifact file
        with open('/kaggle/working/ar_execution_error.log', 'w') as f:
            f.write("FATAL EXCEPTION CAUGHT:\n")
            f.write(error_log)

        # Exit cleanly to force Kaggle to preserve the output directory
        sys.exit(0)
```

With this architecture, the kaggle kernels status poll will return "complete." The orchestrator will then run kaggle kernels output, download the artifacts, and check for the existence of ar_execution_error.log. If the file exists, the orchestrator knows the run failed despite the "complete" status.

The orchestrator then feeds this extracted traceback directly into the LLM Developer Agent. Leveraging the historical context of the codebase and the specific error trace, the LLM generates a surgical patch, overwrites the .ipynb file, and pushes the remediated notebook back to Kaggle, fully satisfying the auto-remediation requirement.[13]

*Note on Hardware Limits:* This try...except pattern effectively captures syntax, logic, and standard runtime exceptions. However, it cannot intercept hardware-level events such as Out of Memory (OOM) GPU kills or hard Kaggle Timeouts (e.g., exceeding 9 hours), which terminate the Python process instantly at the kernel level.[30] To mitigate timeouts, the notebook must include temporal tracking, gracefully breaking out of the training loop and saving partial model

weights just before the strict time limit is reached.[30]

# 6. Component 4: The Audit, Review, and "Roast" Engine

Upon the successful completion of a notebook execution, the resulting code, artifacts, and telemetry must be rigorously evaluated. The user query requests that the system "audit, review and roast it according to the project plan and deliverables." This phase requires advanced multi-agent coordination and specific prompt engineering techniques.

## 6.1. The Psychological and Algorithmic Function of "Roasting"

The explicit instruction to "roast" the notebook is not merely a stylistic choice; it is a highly effective prompt engineering methodology designed to counteract LLM sycophancy. By default, models aligned via Reinforcement Learning from Human Feedback (RLHF) exhibit a strong positivity bias, often providing overly polite, uncritical feedback that overlooks subtle architectural inefficiencies or mathematical flaws.[7]

By forcing the LLM to adopt a hyper-critical, adversarial persona (i.e., a "roast"), the model's latent space is steered toward rigorous fault-finding.[32] This adversarial prompting technique forces the LLM to dissect algorithmic choices, scrutinize hyperparameter gradients, and identify deviations from best practices that a "polite" prompt would gloss over. This technique is utilized in enterprise environments; for instance, Shopify developed an entire open-source tool called "Roast" designed to structure AI workflows for the explicit purpose of critically evaluating and optimizing unit tests at scale.[34]

## 6.2. Architecture of the Reviewer Agent

To implement the Reviewer Agent within AR by HV, the system should mirror the strict parameters utilized by Sakana AI's automated peer review module to ensure maximum fidelity and objectivity [7]:

1. **Frontier Model Selection:** The Reviewer must utilize a flagship model (e.g., GPT-4o or Claude 3.5 Sonnet). Quantized or smaller parameters models lack the deep contextual reasoning required to identify complex data leakage or suboptimal tensor manipulations.[7]
2. **Temperature Control:** The generation temperature must be anchored near zero (e.g., temperature=0.1). "Roasting" code requires highly deterministic, analytical reasoning; high temperatures induce hallucinations and inconsistent evaluations.[7]
3. **Ensemble Reflection:** Rather than relying on a single zero-shot pass, the Reviewer Agent should generate an ensemble of independent critiques (e.g., generating 3 to 5 separate reviews). The agent is then prompted to synthesize these critiques, reflecting on its own analysis to filter out false positives and consolidate a final, highly accurate audit.[7]

## 6.3. The Prompting Framework for the Roast

The orchestrator will construct a massive context prompt containing the current step of the frozen ablation plan, the executed notebook code, and the final telemetry retrieved from W&B. The prompt structure will follow a "Role-Based Template" combined with a "Structured Analysis Template" [5]:

- **Role Directives:** "You are an elite, hyper-critical Principal Machine Learning Architect. Your objective is to brutally roast the provided Kaggle notebook. Do not sugarcoat your findings. Identify every instance of inefficient memory allocation, logical fallacies, data leakage, and suboptimal architectural choices." [32]
- **Plan Adherence:** "Evaluate the code strictly against the required parameters of the frozen ablation plan. Did the code correctly isolate the variables?"
- **Structured Output:** The Reviewer Agent must output its audit as a structured JSON object containing:
  - adherence_score: (0-10) Evaluation of compliance with the ablation plan.
  - critical_flaws: A strict itemization of algorithmic or logical errors.
  - performance_critique: A harsh analysis of the validation metric achieved.
  - actionable_remediations: Specific, diff-ready code blocks required to rectify the identified flaws in the next iteration.

# 7. Component 5: Context-Aware Iterative Notebook Generation

Following the adversarial audit, the pipeline transitions to synthesizing the next version of the notebook. This is a complex synthesis task, as the Developer Agent must simultaneously satisfy the constraints of the *next* step in the frozen ablation plan (Component 1) while incorporating the architectural fixes demanded by the *previous* audit (Component 4).

## 7.1. Context Window and Memory Management

In an autonomous loop, the historical context of previous code versions, execution logs, and audit reports grows exponentially. Indiscriminately feeding this entire history into the LLM context window will inevitably result in token exhaustion and "lost-in-the-middle" attention degradation, where the model forgets early instructions.

AR by HV must implement a rolling, condensed memory mechanism. The Developer Agent tasked with generating version $N+1$ of the notebook receives a highly curated prompt containing:

1. The raw code of notebook version $N$.
2. The specific parameter mutations required by ablation epoch $N+1$.
3. The condensed actionable_remediations array extracted from the Reviewer Agent's

JSON output for version $N$.

The agent is instructed to modify the necessary components (e.g., swapping an activation function or altering a dropout rate) while simultaneously patching the structural anti-patterns identified during the roast.

### 7.2. Diff-Based Code Mutation

Asking an LLM to rewrite an entire 500-line Jupyter notebook from scratch for every iteration is token-inefficient and highly susceptible to regressions (where the LLM accidentally deletes functional boilerplate code).

To ensure stability, the Developer Agent should employ Abstract Syntax Tree (AST) parsing or structured diffing methodologies (similar to the Aider coding assistant integration used in other frameworks [35]). The prompt should instruct the LLM to output specific SEARCH/REPLACE blocks. The AR by HV Python orchestrator will then programmatically apply these diffs to the baseline notebook file. This targeted mutation strategy minimizes token usage and drastically reduces the probability of introducing syntax-breaking hallucinations into previously validated sections of the codebase.[34]

## 8. Component 6: Comprehensive Telemetry Tracking via Weights & Biases

An autonomous research loop is entirely useless if the resulting data, hyperparameter configurations, and model weights are not systematically recorded. The user requirement mandates tracking "everything using weights and biases." Weights & Biases (W&B) is the premier enterprise MLOps platform for experiment tracking and integrates natively with the Kaggle environment.[37]

### 8.1. Programmatic Authentication

Because AR by HV deploys notebooks to Kaggle automatically via the CLI, standard interactive W&B authentication (which requires a user to paste an API key into a browser prompt) is impossible. The system must leverage Kaggle Secrets to authenticate the remote container dynamically.[20]

Before initiating the pipeline, the user must save their W&B API token in the Kaggle account's "Secrets" tab under the label WANDB_API_KEY.[20] The baseline notebook template generated in Component 2 must include the following initialization sequence:

Python

```
from kaggle_secrets import UserSecretsClient
import os
import wandb

# Retrieve and inject the API key securely
user_secrets = UserSecretsClient()
os.environ = user_secrets.get_secret("WANDB_API_KEY")

# Authenticate non-interactively
wandb.login()
```

### 8.2. Tracking Metrics, Configurations, and Artifacts

At the beginning of the execution logic, the notebook must call wandb.init() to create a new W&B Run.[38] Crucially, the parameters from the frozen ablation plan must be injected directly into the config dictionary.[40] This establishes a direct correlation in the W&B dashboard between the specific ablation epoch (e.g., learning_rate=0.001, dropout=0.2) and the final model performance, enabling easy visualization and sorting of experiments.[38]

Throughout the training loop, the notebook utilizes wandb.log() to stream real-time metrics—such as validation loss, training accuracy, learning rate decay, and GPU memory utilization—to the cloud.[38]

Finally, the system must utilize the W&B Artifacts registry. At the conclusion of the training cycle, the generated model weights (.pt or .h5 files) and any Kaggle submission files (submission.csv) must be uploaded via wandb.log_artifact().[38] This practice creates a flawless lineage trace, permanently linking the frozen ablation plan instructions to the executable code, the runtime telemetry, and the final model weights, ensuring total reproducibility.[41]

## 9. Component 7: Automated Documentation Generation (The "Universal Skill")

The final requirement of the AR by HV pipeline is to generate comprehensive documentation for each notebook iteration and its associated audit. This aligns the project with the "Universal Skill" paradigm popularized by Balu Kosuri, which posits that the value of autonomous AI routines is maximized only when their outputs are translated into structured, human-readable technical documentation.[3]

### 9.1. Applying Prompting Techniques for Technical Writing

To automate this phase, the orchestrator will invoke a final LLM persona: the "Summarizer Agent".[12] This agent utilizes prompt chaining to ingest the raw data streams from the execution

pipeline and synthesize them into a polished Markdown document.[42]

Leveraging the "Structured Analysis Templates" and "Role-Based Templates" standard in advanced technical writing prompts [5], the Summarizer Agent receives the ablation plan parameters, the W&B telemetry, and the Reviewer Agent's "roast." The prompt instructs the agent to adopt the persona of a Senior Technical Writer and output a formalized ledger.

## 9.2. Documentation Structure

The generated documentation artifact for each notebook iteration will adhere to a strict structural framework:

- **Experiment ID & Hypothesis:** A plain-English explanation of the specific variables altered in this iteration, derived directly from the frozen ablation plan.
- **Execution Telemetry:** A summary of the runtime environment, including wall-clock execution time, GPU utilization, and final evaluation metrics (e.g., val_bpb or validation loss) retrieved via the W&B API.[40]
- **Adversarial Audit Summary:** A sanitized, professional summary of the Reviewer Agent's "roast." This section will document the critical logical flaws, data leakage issues, or inefficiencies identified during the execution.
- **Iterative Resolution Strategy:** A conclusive statement outlining how the subsequent version of the notebook intends to patch the highlighted issues while progressing to the next step of the ablation matrix.

By generating this documentation autonomously, AR by HV creates a permanent, searchable archive of the entire machine learning research lifecycle. This ensures that the automated trial-and-error process is not merely a black-box computation, but a transparent, mathematically reproducible scientific endeavor.

# 10. Conclusion

The conceptual architecture for "autoresearch by hv" (AR by HV) represents a highly sophisticated, deployable framework for autonomous machine learning experimentation. By synthesizing the rapid iteration mechanics pioneered by Andrej Karpathy's autoresearch, the analytical rigor of adversarial LLM peer review ("roasting"), and the multi-agent cloud orchestration strategies demonstrated in systems like AutoKaggle, AR by HV establishes a robust pipeline for data science discovery.

The feasibility of this system hinges on implementing strict engineering mitigations to navigate the volatility of LLM-generated code and the constraints of remote execution environments. The utilization of a frozen ablation plan is paramount, serving as the definitive bulwark against agentic goal-drift. The integration of Weights & Biases ensures comprehensive telemetry, turning black-box operations into transparent, trackable experiments. Most critically, the engineering workaround to capture Kaggle execution tracebacks via internal file artifacts bypasses fundamental API limitations, enabling the LLM Developer agent to effectively debug

and iteratively improve the codebase. When these architectural constraints are strictly enforced, AR by HV will successfully operationalize a continuous, autonomous pipeline capable of generating optimized machine learning architectures alongside comprehensive, publication-ready documentation.

### Works cited


1. karpathy/autoresearch: AI agents running research on ... - GitHub, accessed April 5, 2026, https://github.com/karpathy/autoresearch
2. A Guide to Andrej Karpathy's AutoResearch: Automating ML with AI Agents | DataCamp, accessed April 5, 2026, https://www.datacamp.com/tutorial/guide-to-autoresearch
3. Cursor for Developer Documentation | by Balu Kosuri - Medium, accessed April 5, 2026, https://medium.com/@k.balu124/cursor-for-developer-documentation-a55e4c17a34e
4. Build a Style Guide Validator with Google's “Opal” for Technical Writers | by Balu Kosuri, accessed April 5, 2026, https://medium.com/@k.balu124/build-a-style-guide-validator-with-googles-opal-for-technical-writers-41ae1191a003
5. 12 Prompting Techniques for Technical Writers | by Balu Kosuri - Medium, accessed April 5, 2026, https://medium.com/@k.balu124/12-prompting-techniques-for-technical-writers-292835e34810
6. The AI Scientist: Towards Fully Automated AI Research, Now Published in *Nature*, accessed April 5, 2026, https://sakana.ai/ai-scientist-nature/
7. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery - GitHub, accessed April 5, 2026, https://github.com/sakanaai/ai-scientist
8. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search - GitHub, accessed April 5, 2026, https://github.com/sakanaai/ai-scientist-v2
9. (PDF) The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery, accessed April 5, 2026, https://www.researchgate.net/publication/383060918_The_AI_Scientist_Towards_Fully_Automated_Open-Ended_Scientific_Discovery
10. Evaluating Sakana's AI Scientist for Autonomous Research: Wishful Thinking or an Emerging Reality Towards 'Artificial Research Intelligence' (ARI)? - arXiv, accessed April 5, 2026, https://arxiv.org/html/2502.14297v2
11. Evaluating Sakana's AI Scientist: Bold Claims, Mixed Results, and a Promising Future?, accessed April 5, 2026, https://isg.beel.org/blog/2025/02/21/sakana-ai-scientist-evaluation/
12. AutoKaggle, accessed April 5, 2026, https://m-a-p.ai/AutoKaggle.github.io/
13. multimodal-art-projection/AutoKaggle - GitHub, accessed April 5, 2026, https://github.com/multimodal-art-projection/AutoKaggle
14. AutoKaggle: LLM powered Multi-Agent framework for solving complex Kaggle

data science competitions | by SACHIN KUMAR | Medium, accessed April 5, 2026, https://medium.com/@techsachin/autokaggle-llm-powered-multi-agent-framework-for-solving-complex-kaggle-data-science-competitions-8ae7d59e40bd
15. ICLR AutoKaggle: A Multi-Agent Framework for Autonomous Data Science Competitions, accessed April 5, 2026, https://iclr.cc/virtual/2025/34847
16. AmirLayegh/agentic-ablation: Automated neural network ablation studies using LLM agents and LangGraph. Systematically remove components, test performance, and gain insights into architecture importance through an intelligent multi-agent workflow. · GitHub, accessed April 5, 2026, https://github.com/AmirLayegh/agentic-ablation
17. ai-scientist-bench/ablation-bench: AblationBench is ... - GitHub, accessed April 5, 2026, https://github.com/ai-scientist-bench/ablation-bench
18. capitalone/ablation: Evaluating XAI methods through ablation studies. - GitHub, accessed April 5, 2026, https://github.com/capitalone/ablation
19. Why LLMs Aren't Scientists Yet: Lessons from Four Autonomous Research Attempts - arXiv, accessed April 5, 2026, https://arxiv.org/html/2601.03315v1
20. Rapid Iteration and Experiment Tracking with WandB - Kaggle, accessed April 5, 2026, https://www.kaggle.com/code/dliend/rapid-iteration-and-experiment-tracking-with-wandb
21. DatawiseAgent: A Notebook-Centric LLM Agent Framework for Adaptive and Robust Data Science Automation - arXiv, accessed April 5, 2026, https://arxiv.org/html/2503.07044v2
22. Public API - Kaggle, accessed April 5, 2026, https://www.kaggle.com/docs/api
23. [Guide] Cron-Schedule Kaggle Notebooks That Need Secrets, accessed April 5, 2026, https://www.kaggle.com/discussions/product-feedback/666571
24. Triggering notebook execution by webhook with Flask and Kaggle API, accessed April 5, 2026, https://www.kaggle.com/discussions/questions-and-answers/397468
25. Error handling - KaggleIngest - Mintlify, accessed April 5, 2026, https://mintlify.com/Anand-0037/KaggleIngest/guides/error-handling
26. Error listing all my kernels from kaggle api cli (error 500) · Issue #178 - GitHub, accessed April 5, 2026, https://github.com/Kaggle/kaggle-api/issues/178
27. Retrieve a kernel's log · Issue #340 · Kaggle/kaggle-cli - GitHub, accessed April 5, 2026, https://github.com/Kaggle/kaggle-api/issues/340
28. Use kaggle kernel command inside kaggle - Reddit, accessed April 5, 2026, https://www.reddit.com/r/kaggle/comments/16apwaa/use_kaggle_kernel_command_inside_kaggle/
29. notebookcba809c5ce - Kaggle, accessed April 5, 2026, https://www.kaggle.com/code/alexchudnovsky/notebookcba809c5ce
30. Please show the results of failed kernels - Kaggle, accessed April 5, 2026, https://www.kaggle.com/discussions/product-feedback/121714
31. traceback — Print or retrieve a stack traceback — Python 3.14.3 documentation, accessed April 5, 2026, https://docs.python.org/3/library/traceback.html
32. I asked my AI to unrelentlessly roast me | by Soul 404 - Answer not found |

Medium, accessed April 5, 2026, https://medium.com/@RomJusta/i-asked-my-ai-to-unrelentlessly-roast-me-c03788952307
33. [LESSON LEARNED] Building CustomGPT based on RoastMe Subreddit : r/ChatGPTPro, accessed April 5, 2026, https://www.reddit.com/r/ChatGPTPro/comments/19bmupy/lesson_learned_building_customgpt_based_on/
34. Introducing Roast: Structured AI workflows made easy (2025) - Shopify Engineering, accessed April 5, 2026, https://shopify.engineering/introducing-roast
35. Evaluating Sakana's AI Scientist for Autonomous Research: Wishful Thinking or an Emerging Reality Towards 'Artificial Research Intelligence' (ARI)? - arXiv, accessed April 5, 2026, https://arxiv.org/html/2502.14297
36. An Evaluation of Sakana's AI Scientist for Autonomous Research: Wishful Thinking or an Emerging Reality Towards 'Artificial General Research Intelligence' (AGRI)? - arXiv, accessed April 5, 2026, https://arxiv.org/html/2502.14297v1
37. Weights and Biases Integration Kernel - Kaggle, accessed April 5, 2026, https://www.kaggle.com/general/318745
38. Experiment Tracking with Weights and Biases - Kaggle, accessed April 5, 2026, https://www.kaggle.com/code/ayuraj/experiment-tracking-with-weights-and-biases/notebook?scriptVersionId=61346071
39. A detailed notebook on experiment tracking - Kaggle, accessed April 5, 2026, https://www.kaggle.com/code/titanpointe/a-detailed-notebook-on-experiment-tracking
40. How to Implement Weights & Biases for ML Tracking - OneUptime, accessed April 5, 2026, https://oneuptime.com/blog/post/2026-01-25-weights-and-biases-ml-tracking/view
41. How to Manage Models with W&B Model Registry and W&B Launch - Wandb, accessed April 5, 2026, https://wandb.ai/vincenttu/enterprise_model_management_wandb/reports/How-to-Manage-Models-with-W-B-Model-Registry-and-W-B-Launch--VmlldzO2OTA5MDMz
42. Prompt Chaining in AI Development - Mirascope, accessed April 5, 2026, https://mirascope.com/blog/prompt-chaining
43. Import and export data - Weights & Biases Documentation - Wandb, accessed April 5, 2026, https://docs.wandb.ai/models/track/public-api-guide